

米点 MODBUS-TCP 六维力传感器通讯协议

版本: v2.0 | 日期: 2026年7月 | 接口: Modbus TCP (端口 502)

修订记录

版本	日期	修订内容
v2.0	2026.07	Modbus-TCP初始版本

1 概述

本文档定义了中科米点六维力传感器基于 Modbus-TCP 协议的上位机通信规范。传感器作为 Modbus TCP Server，默认监听端口 **502**，支持多客户端并发连接。

协议范围说明

Modbus-TCP 通道负责以下功能：

- 控制指令（停止发送、开始转换、单次触发）
- 传感器清零与零点偏移管理
- 设备信息查询（序列号、固件版本）
- 状态监控（系统状态、通信错误计数、过载保护）
- 数据输出（矩阵滤波数据 kg/N、原始数据 mV）
- 网络参数配置（IP、子网掩码、网关等）

1.1 数据类型与字节序

六维力传感器同时测量三个方向的力 (Fx, Fy, Fz, 单位 N) 和三个方向的力矩 (Mx, My, Mz, 单位 N·m)。

数据类型	占用寄存器	字节序	示例
uint16	1 个寄存器	大端 (Big-Endian)	0x0001 = 1
float32	2 个寄存器	小端 (Little-Endian)	1.0 = [0x3F80, 0x0000]
ASCII 字符串	每寄存器 2 字符	顺序存放	"AB" = [0x4142]

2 Modbus-TCP 帧格式

Modbus-TCP 在标准 Modbus RTU 帧前增加 7 字节 MBAP 头，去除 CRC 校验（由 TCP 层保证数据完整性）：

偏移	字段	长度	说明
0	事务标识符 (Transaction ID)	2 字节	主站生成，从站原样返回，用于匹配请求/响应
2	协议标识符 (Protocol ID)	2 字节	固定为 0x0000 (Modbus 协议)
4	长度 (Length)	2 字节	后续字节数 (单元标识 + 功能码 + 数据)
6	单元标识 (Unit ID)	1 字节	固定为 0x01 (传感器设备标识)
7	功能码	1 字节	同 Modbus RTU (0x03 / 0x06 / 0x10 等)
8+	数据	变长	与 Modbus RTU 相同

示例 — 读保持寄存器 (功能码 0x03，读 1 个寄存器):

```
Modbus RTU:          01 03 00 01 00 01 D5 CA
Modbus-TCP MBAP + RTU: 00 01 00 00 00 06 01 03 00 01 00 01
                        |--事务ID--| |协议| |-长度-| |U| |FC| |-地址-| |-数量-|
```

关键区别:

- TCP 帧无 CRC，由 TCP/IP 层保证完整性
- 增加事务 ID 支持并发请求/响应匹配
- 单元标识固定 0x01 (单设备时无实际路由作用)

3 寄存器定义

寄存器按功能分为五大类。所有寄存器均通过 Modbus 功能码访问:

功能码	用途
0x03	读保持寄存器
0x06	写单个寄存器
0x10	写多个寄存器

3.1 控制指令

控制指令寄存器采用**写触发**机制：传感器检测到 Modbus 写操作事件（功能码 0x06）即执行对应动作，不关心写入的具体值。触发后寄存器自动归零。

上位机约定：每次写入 0x0001 作为标准触发值。

地址	名称	说明	默认值	读写	触发机制
0x0001	停止发送	写任意值停止数据主动发送	0x0000	W	写事件触发，触发后自动归零
0x0002	开始数据转换	写任意值启动连续数据转换	0x0000	W	写事件触发，触发后自动归零
0x0003	单次转换	触发单次转换	—	W	写事件触发

3.1.1 停止发送 (0x0001)

停止传感器主动数据发送。写入后传感器立即停止连续数据输出，回到静默状态。

属性	值
寄存器地址	0x0001
访问类型	只写 (W)
触发机制	写事件触发（检测功能码 0x06）
触发后行为	停止数据发送，寄存器值自动恢复为 0x0000
上位机约定	每次写入 0x0001
重试说明	通信超时重发会重复触发，无副作用（已停止状态下再次停止仍为停止）

通信示例：

```
请求：00 02 00 00 00 06 01 06 00 01 00 01
      |--TID--| |-PID-| |-LEN-| |U| |FC| |ADDR| |-VAL-|
响应：00 02 00 00 00 06 01 06 00 01 00 01 （原样返回）
```

3.1.2 开始数据转换 (0x0002)

启动传感器连续数据转换和主动发送。写入后传感器开始按设定频率持续推送六维力数据。数据格式由 0x0032 决定，默认上电为 N 格式。数据内容已扣除零点偏移（mV 格式除外）。

属性	值
寄存器地址	0x0002
访问类型	只写 (W)
触发机制	写事件触发（检测功能码 0x06）
触发后行为	启动连续转换和发送，寄存器值自动恢复为 0x0000
上位机约定	每次写入 0x0001

通信示例（触发指令）：

```
请求：00 03 00 00 00 06 01 06 00 02 00 01
响应：00 03 00 00 00 06 01 06 00 02 00 01 （原样返回，触发成功）
```

触发后，传感器持续推送的数据帧格式（等同于不断返回 0x0033 的读结果）：

```
|-- MBAP Header --|  |-- 读响应 --|  |----- 6 通道 float32 LE (N / N·m) -----
--|

00 XX 00 00 00 1B  01 03 18  Fx_H Fx_L Fy_H Fy_L Fz_H Fz_L  Mx_H Mx_L My_H My_L
Mz_H Mz_L
```

段	值	说明
TID	00 XX	帧序号，每帧递增
PID	00 00	协议标识，固定 0x0000
LEN	00 1B	长度 = 27 (UID + FC + 24 数据)
UID	01	单元标识
FC	03	功能码（读保持寄存器响应）
BYTE_CNT	18	数据字节数 = 24
Fx~Mz	24 字节	6 个 float32 LE，单位由 0x0032 决定。 已扣除零点偏移（mV 格式除外）。

停止连续发送：写 0x0001（停止发送），传感器立即停止数据推送。

3.2 传感器清零与零点偏移

输出值计算公式

F_out[i] = F_raw[i] - F_zero[i] (i = Fx, Fy, Fz, Mx, My, Mz)

- `F_raw`：传感器原始采集值（经矩阵解耦后）
- `F_zero`：存储的零点偏移值，可读可写
- `F_out`：输出给上位机的最终值（数据寄存器 0x0033~0x003E 中返回的值）

关键理解：当 `F_zero` 全部为 0 时，`F_out = F_raw`，输出值 = 原始值，即"未去皮"状态。

3.2.1 自动清零 — 触发寄存器 (0x0030)

触发自动清零后，传感器采集当前 6 个通道的实时值，将其自动存入零点偏移寄存器 (0x0050~0x005B)，后续输出值将减去此零点偏移。

属性	值
寄存器地址	0x0030
访问类型	只写 (W)
触发机制	写事件触发（检测功能码 0x06）
触发后行为	采集当前 6 通道值 → 自动写入 0x0050~0x005B → 寄存器 0x0030 自动恢复为 0x0000
清零后确认	建议读 0x0050~0x005B（12 个寄存器）验证零点偏移是否正确写入

注意：清零前请确保传感器处于零负载状态，否则会产生固定偏置误差。

通信示例（触发清零）：

请求：00 10 00 00 00 06 01 06 00 30 00 01
 |--TID--| |--PID-| |--LEN-| |U| |FC| |ADDR| |--触发器-|
响应：00 10 00 00 00 06 01 06 00 30 00 01

3.2.3 取消清零 — 触发寄存器 (0x0031)

触发后，传感器将 0x0050~0x005B 全部置为 0.0（6 个 float32 零值），此时 `F_zero[i] = 0`，输出值 = 原始采集值，即恢复绝对零点（取消去皮）。

属性	值
寄存器地址	0x0031
访问类型	只写 (W)
触发机制	写事件触发（检测功能码 0x06）
触发后行为	将 0x0050~0x005B 全部置零 → 寄存器 0x0031 自动恢复为 0x0000

通信示例（取消清零）：

请求: 00 11 00 00 00 06 01 06 00 31 00 01
|--TID--| |--PID-| |--LEN-| |U| |FC| |ADDR| |--触发器-|
响应: 00 11 00 00 00 06 01 06 00 31 00 01

3.3 数据输出

数据输出采用"格式选择 + 统一数据区"设计:

- **0x0032 (格式选择)**: 写寄存器切换当前输出单位, 传感器内部完成单位换算, 数据统一映射到 0x0033~0x003E
- **0x0033~0x003E (数据寄存器)**: 只读, 始终返回 12 个寄存器 (6 通道 × 2 寄存器 / float32 LE), 内容由 0x0032 决定

默认上电格式为 N (0x0032 = 2)。kg 和 N 的输出值已扣除零点偏移: $F_out[i] = F_raw[i] - F_zero[i]$ 。mV 为原始电压值, 不扣除零点偏移。

3.3.1 输出格式选择 (0x0032)

写入指定值切换数据输出单位。该寄存器**可读可写**, 读取可获知当前格式。

写入值	格式	说明
0	原始数据 (mV)	数据寄存器返回原始电压值 mV, 未经矩阵解耦, 不扣除零点偏移
1	矩阵滤波数据 (kg)	数据寄存器返回矩阵解耦滤波后的值, 力: kg, 力矩: kg·m, 已扣除零点偏移
2	矩阵滤波数据 (N)	数据寄存器返回矩阵解耦滤波后的值, 力: N, 力矩: N·m, 已扣除零点偏移

通信示例 (切换为 N 格式):

请求: 00 0A 00 00 00 06 01 06 00 32 00 02
|--TID--| |--PID-| |--LEN-| |U| |FC| |ADDR| |VAL=2|
响应: 00 0A 00 00 00 06 01 06 00 32 00 02

读取当前格式:

请求: 00 0B 00 00 00 06 01 03 00 32 00 01
响应: 00 0B 00 00 00 05 01 03 02 00 02 → 当前格式 = N

通道	数据类型	说明
Fx	float32 LE	Fx 力, 单位由 0x0032 决定
Fy	float32 LE	Fy 力, 单位由 0x0032 决定
Fz	float32 LE	Fz 力, 单位由 0x0032 决定
Mx	float32 LE	Mx 力矩, 单位由 0x0032 决定
My	float32 LE	My 力矩, 单位由 0x0032 决定
Mz	float32 LE	Mz 力矩, 单位由 0x0032 决定

通信示例（当前格式 = N，读数据）：

```
请求: 00 0c 00 00 00 06 01 03 00 33 00 0c
响应: 00 0c 00 00 00 1b 01 03 18
      Fx_hi Fx_lo Fy_hi Fy_lo Fz_hi Fz_lo
      Mx_hi Mx_lo My_hi My_lo Mz_hi Mz_lo
      (24 字节 = 6 float32 LE, 单位由 0x0032 决定)
```

三种格式的数据关系：

```
原始 mV → 经标定矩阵解耦 → 滤波 → N 数据（已扣零点偏移）
N 数据 ÷ 9.80665 → kg 数据（已扣零点偏移）
```

4 寄存器总览表

地址	名称	分类	类型	读写	章节
0x0001	停止发送	控制指令	uint16	W	3.1.1
0x0002	开始数据转换	控制指令	uint16	W	3.1.2
0x0030	清零触发	清零偏移	uint16	W	3.2.1
0x0031	取消清零	清零偏移	uint16	W	3.2.3
0x0032	输出格式选择	数据输出	uint16	R/W	3.3

5 常见问题 (FAQ)

Q: 写触发指令时，上次写入的值就是 0x0001，再写 0x0001 还有效吗？

A: 有效。传感器检测的是 Modbus 写操作事件（功能码 0x06），而不是寄存器值的变化。每次写操作都会触发对应动作，触发后寄存器值自动归零。上位机每次固定写入 0x0001 即可。

Q: 通信超时后重发指令会导致重复触发吗？

A: 会。传感器无法区分"新指令"和"超时重发"，每次收到写事件都会执行。建议在应用层通过事务 ID 去重，或在确认前一次已生效后再发下一条。

Q: 标定矩阵如何更新？

A: 标定矩阵（解耦矩阵）的读写由厂家通过 **RS-485 通道**完成，不在 Modbus-TCP 协议中体现。

Q: 如何"取消去皮"（恢复绝对零点）？

A: 写 `0x0031` 触发取消清零，传感器自动将 `0x0050~0x005B` 全部置零，此时 `F_zero[i] = 0`，输出值 = 原始采集值。

Q: kg 和 N 格式有什么区别？如何切换？

A: kg 和 N 均为矩阵解耦滤波后的六维力数据（已扣除零点偏移），仅单位不同。两者共用同一组数据寄存器 `0x0033~0x003E`，通过写 `0x0032` 切换：写 `1` → kg 格式，写 `2` → N 格式（默认）。转换关系： $\text{kg 值} = \text{N 值} \div 9.80665$ 。

Q: 过载阈值设置了，传感器会触发硬件保护吗？

A: 不会。过载阈值仅用于软件告警（系统状态寄存器 bit3），不触发硬件保护。

Q: 网络参数修改后什么时候生效？

A: 网络参数（IP、子网掩码、网关、MAC）修改后需重启传感器才能生效。

Q: TCP 连接断开后数据发送会停止吗？

A: 不会自动停止。传感器作为 TCP Server 持续发送数据，不受单个客户端断开影响。如需停止发送，请写 `0x0001` 寄存器。

Q: 多个客户端同时连接可以吗？

A: 支持。传感器作为 Modbus TCP Server 支持多客户端并发连接，每个客户端独立获取数据。